



Security Audit

1001 Squares of NFT (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	11
Intended Smart Contract Behaviours	12
Code Quality	14
Audit Resources	14
Dependencies	14
Severity Definitions	15
Audit Findings	16
Centralisation	36
Conclusion	37
Our Methodology	38
Disclaimers	40
About Hashlock	41

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The 1001 Squares of NFT team partnered with Hashlock to conduct a security audit of their solana-payment-contract repository. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

1001 Squares of NFT allows users to link and showcase their NFTs on unique grids for a specified time, with ranking determined by the amount paid per day. NFTs are linked through their marketplace URLs, with no smart contract logic for the NFTs themselves, only for processing payments to ensure transparent transactions. By adding NFTs, users can earn the platform's native token, \$DONE, as rewards. This innovative approach provides widespread exposure for NFT creators and collectors.

Project Name: 1001 Squares of NFT

Compiler Version: Rust v1.81.0

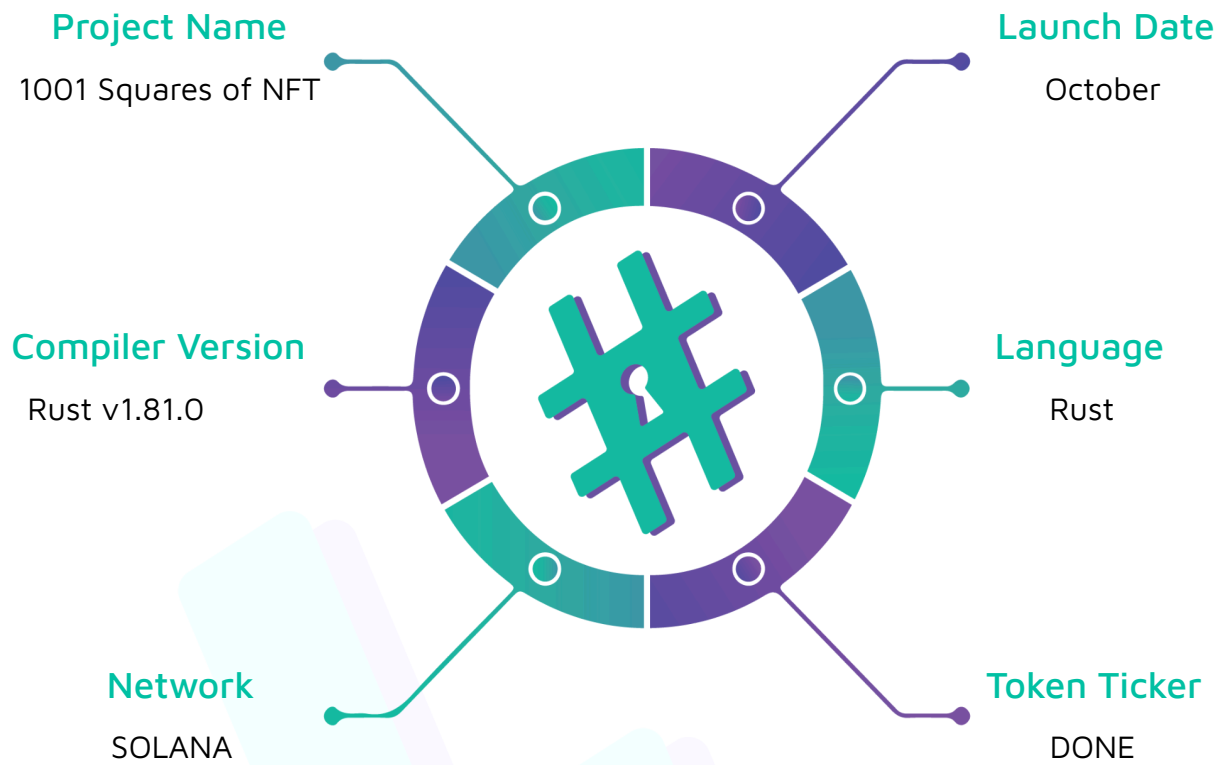
Website: <https://1001squaresofnft.com>

Logo:

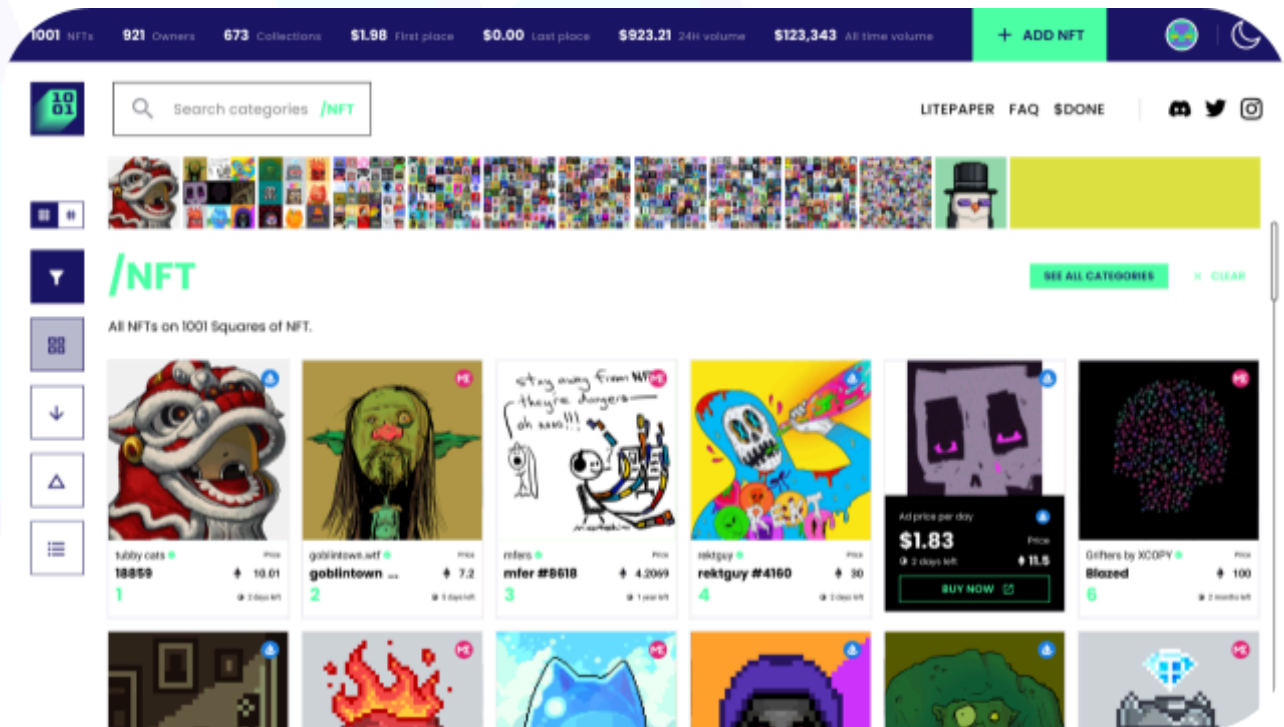
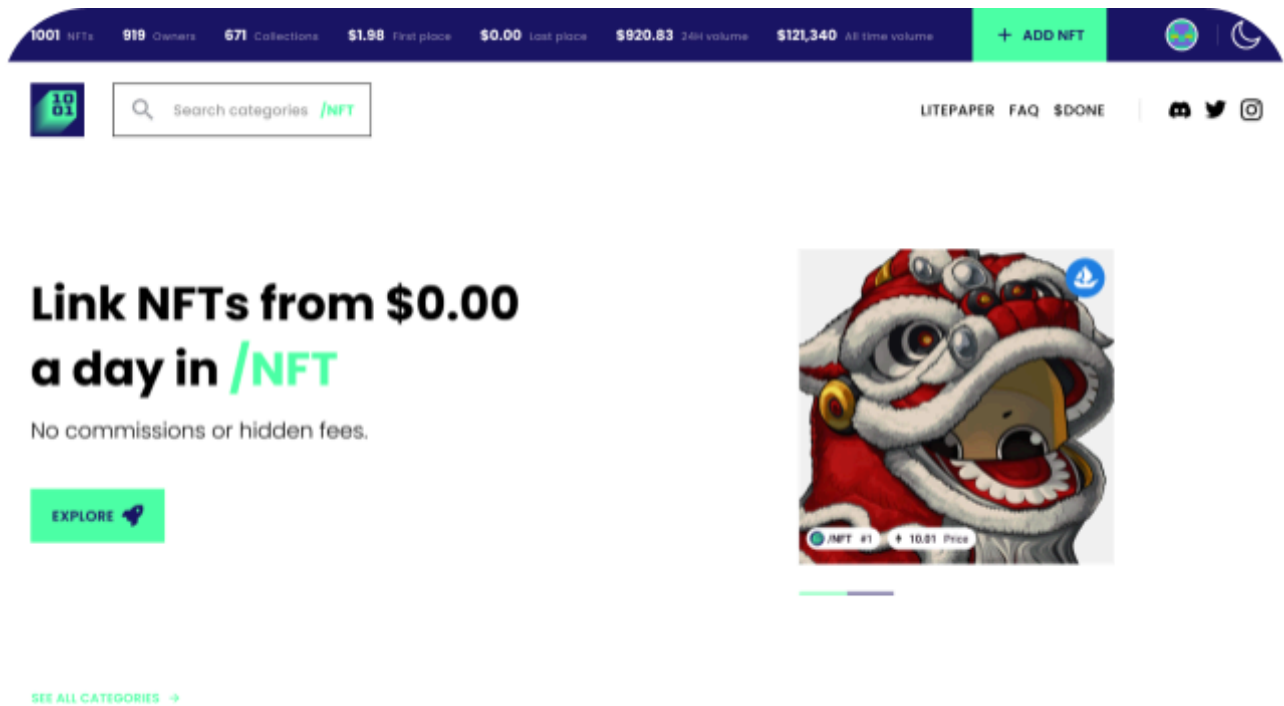


#Hashlock.

Hashlock Pty Ltd

Visualised Context:

Project Visuals:



#Hashlock.

Hashlock Pty Ltd

Audit scope

We at Hashlock audited the rust code within the 1001 Squares of NFT project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	1001 Squares of NFT Protocol Smart Contracts
Platform	Solana / Rust
Audit Date	September, 2024
Contract 1	constants.rs
Contract 1 MD5 Hash	c07f8127ee92a2f38bdb0efd6123eac9
Contract 2	errors.rs
Contract 2 MD5 Hash	69dee73fa116be32dbd2ec1817ec9cd8
Contract 3	lib.rs
Contract 3 MD5 Hash	059ad6248a6d7fd98a32b315ac5ab495
Contract 4	events/create_payment.rs
Contract 4 MD5 Hash	a545c0b0d26c325d9f2544b5bb5e043f
Contract 5	events/create_payment_by_done.rs
Contract 5 MD5 Hash	e963cfd1cab840a93e13ebeaa9645b3a
Contract 6	events/deposit_done_token.rs
Contract 6 MD5 Hash	98c574e230485e8c2b2f0d923e264628
Contract 7	events/deposit_sol.rs
Contract 7 MD5 Hash	9099ea4db0a22bb7329734d1058c8882
Contract 8	events/init_sender_ata.rs
Contract 8 MD5 Hash	cc3cd33db7d79d77e2c400468465d918

Contract 9	events/init_tx_done_token_volume.rs
Contract 9 MD5 Hash	38530464c43d2fb4938969e3bc112677
Contract 10	events/init_tx_sol_volume.rs
Contract 10 MD5 Hash	28d876fdc3afa6f755811ff4818e484d
Contract 11	events/mod.rs
Contract 11 MD5 Hash	41f3a9be63836658089e4478c53ce25d
Contract 12	events/owner_initialized.rs
Contract 12 MD5 Hash	5be9cd0882ea468fabe3b1451d84a61c
Contract 13	events/set_percent.rs
Contract 13 MD5 Hash	cb1879aa1a521e5c9c9d0aefc251bfcf
Contract 14	events/vault_done_token_initialized.rs
Contract 14 MD5 Hash	78b37e0d543e431398ecbda46bea08a2
Contract 15	events/vault_sol_initialized.rs
Contract 15 MD5 Hash	ecc9f9953b93edbd017abc6ba3861bc6
Contract 16	events/withdraw_done_token.rs
Contract 16 MD5 Hash	a43771faed62b8dc0e2011d3325cac97
Contract 17	events/withdraw_sol.rs
Contract 17 MD5 Hash	4d38a677de34c495cc7c9dfe7e6e1efa
Contract 18	instructions/create_payment.rs
Contract 18 MD5 Hash	8b3749ca7b1aa3e6af1a17d3139c6000
Contract 19	instructions/create_payment_by_done.rs
Contract 19 MD5 Hash	144510fcda79793e8b82ac4005af0a34
Contract 20	instructions/deposit_done_token.rs
Contract 20 MD5 Hash	b51b5cfb9ad768da68842b132e94bb21
Contract 21	instructions/deposit_sol.rs
Contract 21 MD5 Hash	2810dc358d934568c4ddc0a3c032a492

Contract 22	instructions/init_sender_ata.rs
Contract 22 MD5 Hash	d64437ce36ad846fd2f1d95e1dcaba3c
Contract 23	instructions/init_tx_done_token_volume.rs
Contract 23 MD5 Hash	ed088ded8ec34a5bb8a22ee768759d93
Contract 24	instructions/init_tx_sol_volume.rs
Contract 24 MD5 Hash	10b600d63f8096ee6bb4811b8da28cd7
Contract 25	instructions/init_vault_done_token.rs
Contract 25 MD5 Hash	c07cc3f4da1144a5b23b22256b2e4a3c
Contract 26	instructions/init_vault_sol.rs
Contract 26 MD5 Hash	01e29782b05b5a59d2f41eea4a7dc296
Contract 27	instructions/initialize.rs
Contract 27 MD5 Hash	4d41c319d1035bdd03de39f5ae4d61e9
Contract 28	instructions/mod.rs
Contract 28 MD5 Hash	a2c0b8057e64358d66d6954767e242d1
Contract 29	instructions/update_master.rs
Contract 29 MD5 Hash	085c364a1808bdb535496323dcbd831e
Contract 29	instructions/set_percent.rs
Contract 29 MD5 Hash	5465f34005bcfa0546da64ea4743f3d8
Contract 30	instructions/withdraw_done_token.rs
Contract 30 MD5 Hash	3ddc12e85354b8afb05db64d23d2b218
Contract 31	instructions/withdraw_sol.rs
Contract 31 MD5 Hash	58f5ee7a122efd6809448aeb2555e42b
Contract 32	state/item_percent.rs
Contract 32 MD5 Hash	1688197ffb1723d08984bd2b538ee68c
Contract 33	state/master.rs
Contract 33 MD5 Hash	dd7597ca4d93f699314448e589b216b3

Contract 34	state/mod.rs
Contract 34 MD5 Hash	7a8947c7944fe8183d539b0690078efe
Contract 35	state/transaction_done_token_volume.rs
Contract 35 MD5 Hash	b5621defc58cefb815d86602cb952e2a
Contract 36	state/transaction_sol_volume.rs
Contract 36 MD5 Hash	e3affbb2989b2313d84fe33a531763de
Contract 37	state/vault_sol.rs
Contract 37 MD5 Hash	c3fbea10b3d45c546da05e38e0514c87
GitHub Commit Hash	5228e6b5b337a69d35e7ae7013fe29b7d889edad
Fix Review Commit Hash	924974a4e4f9dfb01eca6f3bd6f8df0480b0e1a4

Security Rating

After Hashlock's audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

1 High-severity vulnerabilities

5 Medium-severity vulnerabilities

7 Low-severity vulnerabilities

11 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
lib.rs Serves as the entry point.	Program achieves this functionality.
constants.rs Includes the constants that are used by the programs.	Program achieves this functionality.
errors.rs Includes the errors that are used by the programs.	Program achieves this functionality.
events/* Include the events that are used by the programs.	Programs achieve this functionality.
state/* Include the state that is used by the programs.	Programs achieve this functionality.
instructions/create_payment.rs Allows users to create payments, where: - They provide the item id - They specify the amount of SOL they will pay - They get DONE tokens in return by swapping a percentage of SOL to DONE An event is emitted that tracks this information.	Program achieves this functionality.
instructions/create_payment_by_done.rs Allows users to create payments, where: - They provide the item id - They specify the amount of DONE token they will pay - They get a percentage of DONE tokens in return An event is emitted that tracks this information.	Program achieves this functionality.
instructions/deposit_*.rs	Programs achieve this

Allows users to deposit SOL/DONE, where: - They provide the amount The information is only handled by an event.	functionality.
instructions/init_sender_ata.rs Creates the Associated Token Account (ATA) for the user.	Program achieves this functionality.
instructions/init_tx_*.rs Create the transaction SOL/DONE volume Account for the user.	Programs achieve this functionality.
instructions/init_vault_*.rs Create the vault of SOL/DONE, where payments and deposits are stored.	Programs achieve this functionality.
instructions/initialize.rs Creates the master Account with initial values.	Program achieves this functionality.
instructions/update_master.rs The admin can: - Change the percentages of the master Account. - Set the new owner.	Program achieves this functionality.
instructions/withdraw_*.rs Allow the admin to withdraw SOL/DONE.	Programs achieve this functionality.

Code Quality

This audit scope involves the smart contracts for the 1001 Squares of NFT project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the 1001 Squares of NFT smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external program calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] initialize - Initialization can be frontrun, leading to ownership takeover

Description

The `initialize` instruction in the contract can be invoked by any account. It assigns the caller as the owner of the master account, which is the administrative authority of the program.

Vulnerability Details

There is no mechanism to restrict access to this instruction, such as validating that only the deployer of the program or an authorised entity can perform the initialization.

```
pub fn process(ctx: Context<InitializeCtx>, percent_pay_w_sol: u16,
percent_pay_w_done_token: u16) -> Result<()> {

    let master = &mut ctx.accounts.master;

    let signer = &ctx.accounts.signer;

    require!(percent_pay_w_sol <= 10000, CustomErrors::InvalidPercent);

    require!(percent_pay_w_done_token <= 10000, CustomErrors::InvalidPercent);

    if master.is_initialized {

        return Err(CustomErrors::MasterAccountAlreadyInitialized.into());

    }

    master.is_initialized = true;

    master.owner = signer.key();
```

```
master.percent_pay_w_sol = percent_pay_w_sol;  
  
master.percent_pay_w_done_token = percent_pay_w_done_token;  
  
emit!(OwnerInitialized {});  
  
Ok(())  
}
```

Impact

A malicious actor can take the complete ownership of the master account and hence the program.

Recommendation

Implement access control on the caller of the `initialize` instruction. A common practice is to use the program's `upgrade_authority` as the authorized address to call the `initialize` function if the program has an upgrade authority. Another solution is to set a hardcoded address of the expected owner's public key to restrict access to only that specific address.

Status

Resolved

Medium

[M-01] initialize,set_percent - Lack of input validation of percent can lead to draining of DONE token vault

Description

The owner of the program sets and updates the percent in the `initialize` and `set_percent` instructions respectively, that is used for DONE token returns in the `create_payment` instruction, depending on the amount of SOL that users paid.

Vulnerability Details

The `percent` parameter is of type `u64`, allowing values between `0` and `2^64 - 1`. Without a validation mechanism to constrain the `percent` field, an excessively high value could be set.

```
pub fn process(ctx: Context<InitializeCtx>, percent_pay_w_sol: u16,
percent_pay_w_done_token: u16) -> Result<()> {

    let master = &mut ctx.accounts.master;

    let signer = &ctx.accounts.signer;

    require!(percent_pay_w_sol <= 10000, CustomErrors::InvalidPercent);
    require!(percent_pay_w_done_token <= 10000, CustomErrors::InvalidPercent);

    if master.is_initialized {

        return Err(CustomErrors::MasterAccountAlreadyInitialized.into());

    }

    master.is_initialized = true;

    master.owner = signer.key();
```

```

    master.percent_pay_w_sol = percent_pay_w_sol;

    master.percent_pay_w_done_token = percent_pay_w_done_token;

    emit!(OwnerInitialized {});

    Ok(())
}

```

Impact

The vault of the done token can be completely drained even in one payment, if the percent has been set incorrectly to a very high value.

Recommendation

Add an explicit check to ensure the percent value falls within the expected range, that aligns with the project's requirements.

Note

If the finding **[L-01]** is addressed, the checks should take that change also into account.

Status

Resolved

[M-02] create_payment, create_payment_by_done - One-time limitation of item_id prevents recurring payments

Description

The create_payment and create_payment_by_done instructions both use item_id to create a corresponding payment record. However, each instruction can only be called once for a specific item_id.

Vulnerability Details

Each `item_payment` account is created by using `item_id` and has the `init` constraint.

```
#[account(  
    init,  
    payer = signer,  
    seeds = [ITEM_PAYMENT, item_id.to_le_bytes().as_ref()],  
    bump,  
    space = 8 + ItemPayment::INIT_SPACE,  
)]  
  
item_payment: Account<'info, ItemPayment>,  
...  
  
#[account(  
    init,  
    payer = signer,  
    seeds = [ITEM_PAYMENT, item_id.to_le_bytes().as_ref()],  
    bump,  
    space = 8 + ItemPayment::INIT_SPACE,  
)]  
  
item_payment: Account<'info, ItemPayment>,
```

Impact

Users cannot execute recurring payments for items, since each `item_id` can only be used once. Furthermore, since there is no check which user creates a payment for a specific item, a malicious actor can create a payment for another user's item and as a result, the rightful user cannot pay for their specific item.

Recommendation

Consider allowing payments for a specific `item_id` to be made more than once. This can be achieved by changing `init` constraint to `init_if_needed` and including additional unique identifiers (e.g. `transaction_id`) to differentiate between multiple payments for the same `item_id`.

Status

Resolved

[M-03] `create_payment,create_payment_by_done` - Limitation of unique `item_id` can be bypassed for item payments

Description

Users can create payments for the same `item_id` by paying with SOL and DONE tokens as well. Although there is a limitation of each `item_id` to be used only once, the case of the two different payment methods to take place for the same item is not handled.

Vulnerability Details

The `item_id` is an argument that is given to each payment instruction and with that the `item_payment` is created. By implementing the `init` constraint, the business logic enforces the usage of the `item_id` only once.

```
#[account(  
    init,  
    payer = signer,  
    seeds = [ITEM_PAYMENT, item_id.to_le_bytes().as_ref()],  
    bump,  
    space = 8 + ItemPayment::INIT_SPACE,  
)]  
  
item_payment: Account<'info, ItemPayment>,  
  
#[account(  
    init,
```

```

    init,

    payer = signer,

    seeds = [ITEM_PAYMENT, item_id.to_le_bytes().as_ref()],

    bump,

    space = 8 + ItemPayment::INIT_SPACE,

)]

item_payment: Account<'info, ItemPayment>,

```

Impact

The limitation of the uniqueness of `item_id` for an `item_payment` can be bypassed and create scenarios where 2 simultaneous payments for the same `item_id` take place.

Recommendation

Implement a mechanism that ensures each `item_id` can only be associated with a single payment type by restricting the ability to call both `create_payment` and `create_payment_by_done` for the same `item_id`.

Status

Resolved

[M-04] `create_payment` - Absence of transfers tracking

Description

The instruction `create_payment` lacks mechanisms to track payment details. This is the case with `create_payment_by_done`, `deposit_done_token`, and `deposit_sol` as well, which handle payments and deposits respectively.

Vulnerability Details

The instructions that handle payments update the transaction volume of the user and at the end, an event is emitted.

```
pub fn process(ctx: Context<CreatePaymentContext>, item_id: u64, amount_sol: u64) ->
Result<()> {
    ...

    transaction_sol_volume.amount += amount_sol;

    ...

    emit!(CreatePaymentEvent {
        signer: signer.key(),
        item_id,
        amount: amount_sol,
    });
}
```

Impact

The payments and the deposits of users are not tracked by the program.

Recommendation

Implement mechanisms for recording and tracking individual user payments and deposits.

Status

Acknowledged

[M-05] done-token - Mint and Freeze authorities not renounced

Description

The `done-token.ts` script does not include the functionality for revoking freeze and mint accounts. The script contains comments that indicate the necessary steps after creating the token mint and minting an initial supply of tokens. The post-deployment transactions cannot be guaranteed that will take place. Manual execution of these commands can lead to inconsistencies and oversight, which may compromise the integrity of the token.

```
// run the scripts below by the deployer's wallet  
  
//  
  
// spl-token authorize token_mint_addressss mint --disable  
  
// spl-token authorize token_mint_address freeze --disable
```

Impact

The security and smooth functionality of the token is not guaranteed.

Recommendation

The script should be updated to include the automation of both minting and freezing authority revocations, by making use of the SPL instructions. By integrating these operations directly into the script, it will be ensured that these operations are performed correctly.

Status

Resolved

Low

[L-01] `create_payment` - Lack of precision in `percent` variable

Description

The current implementation of the `create_payment` instruction has a precision limitation. Specifically, the `percent` variable lacks the capability to handle precise decimal values. In order for the calculation to be correct, the `percent` variable has to hold whole number percentages (e.g. 1%) and can't hold percentages with decimals (e.g. 1.1%, 1.01%).

Recommendation

Consider increasing the precision of percentage calculations by using higher precision in `percent` variable. By making this change, the calculation formula on the return of DONE tokens needs to be refactored.

Status

Resolved

[L-02] `create_payment, create_payment_by_done` - `transaction_volume` Account constraints inconsistency

Description

In the `create_payment_by_done` instruction, the `transaction_done_token_volume` account is defined with the `init_if_needed` constraint. This allows the account to be created if it does not exist, ensuring proper initialization. In contrast, the `create_payment` instruction defines the `transaction_sol_volume` account with the `mut` constraint, which requires the account to be pre-existing and initialized and does not allow for account creation.

Recommendation

Standardize the constraints for similar `transaction_volume` accounts across all instructions.

Status

Resolved

[L-03] initialize - Redundant discriminators

Description

There are redundant implementations in the initialization instructions `initialize`, `init_vault_done_token` and `init_vault_sol`. Specifically, these functions include discriminators to check for initialization status that are unnecessary due to `init` constraints in place.

```
if master.is_initialized {  
    return Err(CustomErrors::MasterAccountAlreadyInitialized.into());  
}
```

Recommendation

Remove these redundant checks.

Status

Acknowledged

[L-04] create_payment - Decimal division precision

Description

The `create_payment` instruction calculates `amount_done_token_out` through a series of integer arithmetic operations. The calculation first multiplies `_amount_sol` by a percentage and then divides by a scaling factor based on the token's decimal precision. This approach involves integer division at multiple stages, which truncates any fractional part of the result. As a consequence, when `_amount_sol` is small, since there is no lower limit of the amount of SOL transferred, the final amount of tokens computed for transfer is rounded down to a value that can be lower than intended. This rounding issue leads to underpayment, where users receive fewer tokens than expected.

```
let amount_done_token_out = (_amount_sol * master.percent * 100) / 10000;

let amount_done_token_out = amount_done_token_out / (10u64.pow(9 as u32) /
10u64.pow(mint_of_token_being_sent.decimals as u32));
```

Recommendation

Use higher precision arithmetic methods to manage token amount calculations more accurately.

Status

Acknowledged

[L-05] init_tx_done_token_volume, init_tx_sol_volume - Redundant initialization calls can lead to event spamming

Description

The instructions use the `init_if_needed` constraint for initializing the `transaction_sol_volume` and `transaction_done_token_volume` accounts, respectively. The `init_if_needed` constraint can result in multiple calls to the initialization functions, as it does not enforce a single-time initialization. Consequently, this may lead to excessive and redundant emissions of `InitTxSolVolume` and `InitTxDoneTokenVolume` events, which could clutter transaction logs.

Recommendation

Consider changing the constraint to `init` instead of `init_if_needed`.

Status

Acknowledged

[L-06] done-token - Hardcoded paths and endpoints

Description

The script contains hardcoded paths and API endpoints, which can create potential security and flexibility issues. Specifically, the script includes a fixed path to a secret key

and a hardcoded API endpoint for interacting with the Solana network. For enhanced security and adaptability, these values should be dynamically configured through environment variables.

```
const umi = createUmi(  
  "https://api.devnet.solana.com"  
  // "http://localhost:8899"  
);  
  
const userWallet = umi.eddsa.createKeypairFromSecretKey(  
  new Uint8Array(  
    JSON.parse(  
      fs.readFileSync("/Users/lainhathoang/.config/solana/id.json", "utf-8")  
    )  
  )  
);
```

Recommendation

Replace hardcoded values in the `done-token.ts` script with environment variables. Specifically, use an environment variable for the secret key path to protect sensitive data and for the API endpoint to facilitate easy configuration changes across different environments.

Status

Resolved

[L-07] done-token - Impersistent URI

Description

The URI used for metadata is hosted on GitHub Gist. This URL, which points to a raw JSON file containing token details, is not a stable or permanent storage solution. The reliance on GitHub Gist introduces a risk that the metadata might be altered or deleted.

```
const metadata = {  
  
  name: "DONE Token",  
  
  symbol: "DONE",  
  
  description: "$DONE token description",  
  
  uri:  
  "https://gist.githubusercontent.com/lainhathoang/38463850b65881fcb1d073c4a8f223af/raw/  
b48a6b922ab14cd8547944cbff7ac3fc6f98d32a/done_token.json",  
  
};
```

Recommendation

Utilize a URL that points to a pinned IPFS (InterPlanetary File System) URI. IPFS offers a decentralized and permanent storage mechanism that ensures the content remains accessible and unchanged as long as it is pinned by at least one node.

Status

Resolved

Gas

[G-01] state/master - Inefficient variable type in Master struct

Description

In the `Master` struct, the `percent` variable is currently declared as `u64`. Depending on the intended maximum value of the `percent` field, this variable could potentially be optimized to a smaller integer type, such as `u16` or even `u8`.

Recommendation

Review the expected range of values for `percent`. If it remains within the bounds of a smaller integer type, change the type of `percent` from `u64` to `u16` or `u8`. Additionally, ensure that any references to this variable throughout the codebase are adjusted accordingly to maintain consistency and prevent type mismatch errors.

Note

If the finding [L-01] is addressed, the type of `percent` should be `u16`.

Status

Resolved

[G-02] lib - Inconsistent context struct naming

Description

The codebase includes two deposit functions, `deposit_sol` and `deposit_done_token`, where the context struct names are inconsistent with the function names, `TransferSolCtx` and `TransferTokenCtx` are the struct names respectively. It is common practice to name context structs in a way that closely aligns with the associated function to improve code clarity and maintainability.

Recommendation

Rename the context structs to more closely reflect the function names they are used with, which aligns more closely to Rust's coding guidelines.

Status

Resolved

[G-03] `withdraw_sol` - Inconsistent handling of SOL balances

Description

There are inconsistencies in the methods used for transferring SOL between accounts. The `create_payment` and `deposit_sol` functions utilize `SystemProgram::transfer`, which transfers SOL through the System Program's CPI. In contrast, the `withdraw_sol` function uses low-level operations, specifically `sub_lamports` and `add_lamports`.

Recommendation

Standardize the transfer method across all functions. Refactoring the `withdraw_sol` function to use `SystemProgram::transfer` would ensure uniformity in SOL transfers.

Status

Acknowledged

[G-04] `errors` - Unused error codes

Description

The `CustomErrors` enum includes error codes that are not currently used within the contract's logic. Specifically, the error codes `Unauthorized` and `InvalidPercent` are defined but are not referenced or utilized in any function or error handling mechanism within the codebase.

Recommendation

Remove the unused error codes from the `CustomErrors` enum if they are not needed. However, based on their declaration and naming, it appears that they are intended for access control errors and for validating the setting of the `percent` value.

Status

Resolved

[G-05] `create_payment_by_done, hello` - Unnecessary code present

Description

The `create_payment_by_done` instruction contains a commented-out line of code which is not currently serving any functional purpose. Furthermore, the codebase includes a `hello` instruction, which appears to be an example, and is not relevant to the primary functionality.

Recommendation

Review the commented-out code and remove it if it is not intended for future use. The `hello` instruction and any other non-essential code should be also removed.

Status

Resolved

[G-06] `state/transaction_sol_volume` - Typo in struct name

Description

A typographical error has been identified in the code where the struct `TransctionSolVolume` is used within the `InitTransactionSolVolumeCtx` context.

Recommendation

The correct name of the struct should be `TransactionSolVolume` instead of `TransctionSolVolume`. Update the code to use the correct struct name consistently throughout the codebase.

Status

Resolved

[G-07] create_payment - Redundant field names

Description

The instruction `create_payment` contains redundant field names in its struct initializations. This is also the case for `create_payment_by_done`, and `set_percent` as well. Specifically, the field names used in these initializations are identical to the corresponding variable names.

```
emit!(CreatePaymentEvent {  
  
    signer: ctx.accounts.signer.key(),  
  
    item_id: item_id,  
  
    amount: _amount_sol,  
  
});
```

Recommendation

Use the shorthand syntax for struct initialization, where the field name and variable name are identical.

Status

Resolved

[G-08] create_payment - Unnecessary casting

Description

The calculation for `amount_done_token_out` includes unnecessary casting of integers to `u32`. Specifically, the code casts `9` and `mint_of_token_being_sent.decimals` to `u32` before using them in the `pow` method for `u64`. This casting is redundant as these values are integers and will be correctly handled without explicit casting.

```
let amount_done_token_out = amount_done_token_out / (10u64.pow(9 as u32) /
10u64.pow(mint_of_token_being_sent.decimals as u32));
```

Recommendation

Remove the casting, since it does not serve a purpose. The revised code will be clearer and more straightforward, maintaining the same functionality.

Status

Resolved

[G-09] create_payment_withdraw_done_token - Redundant use of as_ref Method

Description

The `as_ref` method is redundantly applied to `TOKEN_ACCOUNT_OWNER` when constructing the seeds array for the `CpiContext::new_with_signer` method.

```
let seeds = &[TOKEN_ACCOUNT_OWNER.as_ref(), &[bump]];
```

Recommendation

Remove the redundant `as_ref()` call. This change will simplify the code and clarify that `TOKEN_ACCOUNT_OWNER` is already a byte slice.

Status

Resolved

[G-10] create_payment_by_done - Redundant check for negative values

Description

The instruction `create_payment_by_done` contains a check to validate that the `_amount_done` parameter is greater than zero. The variable `_amount_done` is `u64` which means that it cannot be lower than zero. The condition should check if `_amount_done` is

equal to zero. This is also the case for `deposit_done_token`, `deposit_sol`, `withdraw_done_token`, and `withdraw_sol`.

```
if _amount_done <= 0 {  
    return Err(CustomErrors::InvalidAmount.into());  
}
```

Recommendation

Adjust the conditions and eliminate unnecessary checks that do not contribute to the validation process.

Status

Resolved

[G-11] done-token - Incorrect console.log

Description

The script contains an incorrect `console.log` statement. Specifically, after executing the `createAndMint` function, which mints `1,000,000,001` tokens, the script logs a message stating, "Successfully minted 1 million tokens." This log message inaccurately reflects the number of tokens minted, which is `1,000,000,001`, not `1,000,000`.

Recommendation

Define a variable for the mint amount and use it consistently throughout the script. This approach will ensure that the logged message always reflects the actual number of tokens minted.

Status

Resolved

Centralisation

The 1001 Squares of NFT project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the 1001 Squares of NFT project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#Hashlock.

#Hashlock.

Hashlock Pty Ltd